

Etapa 1 — Revisão bibliográfica (35 minutos)

Árvore Binária de Busca (ABB): Cada nó contém uma chave; todos os valores da subárvore esquerda são menores que o nó, e todos os valores da subárvore direita são maiores. Essa restrição de ordenação faz com que cada busca seja uma descida guiada a partir da raiz, semelhante à busca binária em um vetor ordenado. Em caso balanceado, busca, inserção e remoção são $O(\log n)$; mas se as chaves forem inseridas em ordem, a árvore degenera em uma lista encadeada, com pior caso $O(n)$.

Árvore AVL: É uma árvore binária de busca auto-balanceada em que a diferença de altura entre as duas subárvores de qualquer nó não pode passar de 1. Quando essa diferença é violada, a árvore é rebalanceada por meio de rotações. Busca, inserção e remoção são $O(\log n)$ no caso médio e no pior caso. A AVL é mais rigidamente balanceada que a rubro-negra, o que a torna mais rápida para buscas, mas com custo maior de rotações em inserções/remoções — a remoção pode exigir até $O(\log n)$ rotações no pior caso.

Árvore Rubro-Negra: Também é uma árvore binária de busca auto-balanceada, mas mantém o equilíbrio por meio de cores nos nós (vermelho ou preto). As propriedades principais incluem: a raiz é preta; filhos de nó vermelho devem ser pretos (não há dois vermelhos consecutivos); e todo caminho de um nó até suas folhas contém o mesmo número de nós pretos. Essas regras garantem que o caminho mais longo não seja mais que o dobro do mais curto, com altura máxima de $2 \cdot \log(n+1)$. O balanceamento é mais “frouxo” que o da AVL, exigindo menos rotações em inserção/remoção (no máximo 3 rotações na remoção), mas com buscas ligeiramente mais lentas. É amplamente usada em bibliotecas padrão, como `TreeMap/TreeSet` do Java e `map/multimap` do C++ STL.

Árvore B: É uma árvore de busca multi-caminho em que cada nó pode conter várias chaves e vários filhos. Para uma árvore B de ordem m , cada página tem no máximo m descendentes; exceto a raiz e as folhas, cada página tem pelo menos $\lceil m/2 \rceil$ descendentes; e todas as folhas ficam no mesmo nível. Na inserção, se um nó transborda, ele é dividido e a chave do meio é promovida ao pai, podendo propagar até a raiz e, no pior caso, aumentar a altura em 1. Como cada nó armazena muitas chaves, a altura da árvore é muito baixa — por exemplo, uma árvore B de ordem 512 com 1 milhão de chaves precisa de no máximo 3 acessos para localizar um registro. É adequada para sistemas de armazenamento em disco.

Árvore B+: É uma variação da árvore B em que todos os registros de dados ficam armazenados apenas nas folhas; os nós internos guardam apenas chaves de índice. As folhas são ligadas entre si por ponteiros, o que facilita consultas por intervalo. Os nós não-folha têm estrutura semelhante à ISAM: cada nó (exceto a raiz) tem m chaves ($d \leq m \leq 2d$) e o número de ponteiros é $m+1$. Na inserção, se a folha está cheia, ela é dividida e a menor chave do novo nó é inserida no pai, podendo propagar a divisão para cima. Essa estrutura torna a B+ especialmente eficiente para varreduras de intervalo e acesso sequencial.

Heap: É uma árvore binária completa que satisfaz a propriedade de heap. Em um heap mínimo, o pai é sempre menor ou igual aos filhos. É representado eficientemente em vetor: a raiz fica na posição 1, o pai do nó i é $\lfloor i/2 \rfloor$, o filho esquerdo é $2i$ e o direito é $2i+1$. Inserção e remoção do mínimo/máximo são $O(\log n)$. É a implementação clássica de filas de prioridade, usada nos algoritmos de Dijkstra, Prim e A*.

Trie (árvore de prefixos): É uma árvore em que os nós se ramificam por caractere, e cada nó pode ter até 256 direções (correspondentes à tabela ASCII). O caminho da raiz até um nó corresponde a um prefixo, e um marcador booleano no nó indica se aquele prefixo é uma palavra completa. A busca de uma palavra de comprimento k é $O(k)$, independente do tamanho do dicionário. Suporta naturalmente consultas por prefixo — algo que tabelas hash não fazem. A desvantagem é o grande desperdício de espaço (cada nó precisa de muitos ponteiros vazios).

Etapa 2 — Quadro comparativo (35 minutos)

Estrutura Organização dos dados

Regra ou propriedade principal

Operação ou ajuste importante

Exemplo de aplicação

Referência consultada

ABB Esquerda menor, direita maior Todos os valores da subárvore esquerda $<$ nó $<$ subárvore direita Busca é descida guiada; se desbalanceada, degenera para $O(n)$ Mapeamentos/conjuntos ordenados; percurso em ordem gera sequência ordenada

AVL Esquerda menor, direita maior + equilíbrio de altura Diferença de altura entre subárvores de qualquer nó ≤ 1 Rotações; remoção pode exigir até $O(\log n)$ rotações no pior caso Aplicações com muitas buscas; busca mais rápida que rubro-negra

Rubro-negra Esquerda menor, direita maior + restrições de cor Raiz preta; filhos de vermelho são pretos; mesmo número de pretos

em todos os caminhos Recoloração + rotações; no máximo 3 rotações na remoção TreeMap/TreeSet do Java; map do C++ STL B Multi-caminho balanceada, todas as folhas no mesmo nível Exceto raiz/folhas, cada nó tem pelo menos $\lceil m/2 \rceil$ descendentes; nó contém várias chaves Divisão do nó quando cheio, com promoção da chave do meio Sistemas de armazenamento em disco; índices de banco de dados B+ Multi-caminho balanceada, dados apenas nas folhas Folhas ligadas entre si; nós internos apenas com chaves de índice Divisão da folha com chave mínima promovida ao pai; suporta varredura por intervalo Índices de banco de dados; consultas por intervalo Etapa 3 — Identificação por analogias (35 minutos)

1. Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro. Estrutura: Árvore AVL. Justificativa técnica: A AVL exige que a diferença de altura entre as duas subárvores de qualquer nó não passe de 1, e restaura o equilíbrio por meio de rotações. É exatamente o mecanismo de “rotação” que mantém o equilíbrio rigoroso. Limite da analogia: A analogia da “estante” sugere espaço físico limitado, mas as rotações da AVL apenas alteram os ponteiros entre nós, sem mover dados nem rearranjar espaço.
2. Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida. Estrutura: Árvore B (ou B+). Justificativa técnica: Nós de uma árvore B podem conter várias chaves; quando um nó transborda, ele é dividido em dois e a chave do meio é promovida ao pai. A divisão das folhas da B+ segue a mesma lógica. Limite da analogia: A analogia da “página” sugere blocos físicos de tamanho fixo, mas a divisão na árvore B é uma operação lógica, e o tamanho dos nós resultantes depende da implementação e da ordem, não necessariamente de limites físicos de página.
3. Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro. Estrutura: Heap (fila de prioridade). Justificativa técnica: Em um heap mínimo, o pai é sempre menor que os filhos; logo, a raiz é sempre a de maior prioridade (o menor valor), podendo ser consultada em $O(1)$ e removida em $O(\log n)$. Limite da analogia: A analogia sugere que o “topo” é uma posição física, mas a raiz do heap é logicamente o primeiro elemento do vetor; após a remoção, o último elemento preenche a raiz e é ajustado por “descida”, não é simplesmente “retirar do topo”.
4. Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo. Estrutura: Trie (árvore de prefixos). Justificativa técnica: Na Trie, o caminho da raiz até um nó corresponde a um prefixo, e palavras com o mesmo prefixo compartilham o mesmo trecho de caminho; um marcador no nó indica se ele é uma palavra completa. Limite da analogia: A analogia sugere que as “letras” são naturalmente compartilhadas, mas cada nó da Trie precisa manter ponteiros para todos os caracteres possíveis, desperdiçando espaço mesmo quando o prefixo não existe; implementações reais usam tries de busca ternária para mitigar isso.
5. Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca. Estrutura: Árvore Rubro-Negra. Justificativa técnica: A rubro-negra usa restrições de cor (raiz preta, filhos de vermelho pretos, mesma altura preta em todos os caminhos) para garantir que o caminho mais longo não passe do dobro do mais curto. Após inserção/remoção, restaura as propriedades com recoloração e rotações. Limite da analogia: A “cor” é puramente uma marcação abstrata, sem qualquer informação semântica; sua única função é codificar a restrição de comprimento dos caminhos, não representar alguma propriedade dos dados.
6. Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo. Estrutura: Árvore B+. Justificativa técnica: Na B+, todos os registros ficam nas folhas, e as folhas são ligadas por ponteiros, de modo que uma varredura por intervalo só precisa localizar a folha inicial e percorrer a cadeia. Limite da analogia: A analogia sugere que a “ligação” é um ponteiro físico, mas em implementações em disco pode ser um endereço de bloco; além disso, a eficiência da consulta por intervalo também depende do fator de preenchimento das folhas, o que a analogia não captura.

7. Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita. Estrutura: Árvore Binária de Busca (ABB). Justificativa técnica: A propriedade definidora da ABB é exatamente que todos os valores da subárvore esquerda são menores que o nó e os da direita são maiores. Limite da analogia: A analogia sugere que a estrutura se mantém naturalmente balanceada, mas uma ABB comum degenera em lista encadeada quando as inserções são ordenadas; a ABB em si não contém nenhum mecanismo de balanceamento.